DESIGN & IMPLEMENTATION PLAN · 9 JULY 2026

FastBake

One click from a high-poly sculpt or scan to a game-ready asset: watertight merge → AutoRemesher quad retopo → seam-aware UVs → baked, channel-packed textures — entirely inside Blender.

Target: Blender 4.5 LTS → 5.2 **Engine:** AutoRemesher 1.0 (MIT)

Research: 6 domain reports, adversarially fact-checked

PREFLIGHT	MERGE	RETOPO	UV	BAKE
validate · transforms · re-bake	watertight union	AutoRemesher 1.0	seams + SLIM	normal · color · rou

1 The opportunity
2 Request analysis
3 Competitive landscape
4 Architecture
5 Stage-by-stage design
6 Naming engine
7 UI & UX
8 Presets
9 Packaging & platform
10 Risks
11 Roadmap
12 Open decisions

SECTION 1

The opportunity – and a lucky week to start

> VERIFIED 2026-07-09
>
> **AutoRemesher 1.0.0 stable shipped three days ago** (6 July 2026), ending five years of dormancy. It was **relicensed from GPLv3 to MIT**, gained a headless CLI (`--target-quads` , `--adaptivity` , `--sharp-edge` , `--smooth-normal` , `--edge-scaling`) and a Qt-free C++ core with a **native progress callback**. The repo is actively maintained again (last push 8 July). A community fork already proves the cleanest integration path: the core compiled as a 1–3 MB Python wheel bundled inside a Blender extension.

The market intersection FastBake targets is **empty as of mid-2026**. Every strong baker (SimpleBake, Bake Wrangler, Marmoset, Substance) starts from an existing low-poly with UVs. Every strong retopo tool (QuadRemesher, QRemeshify, AutoRemesher's two new wrapper add-ons) stops before UVs and baking. The in-Blender full-chain attempts are abandoned (AssetGen), decimation-based (Assetify), or thin and mistrusted (Soloware, $99). Blender's own baking-workflow revamp is officially deferred past 2026, so the native pain points — manual image-node juggling, silent data loss, no packing, no naming — stay open through at least 2027.

Nobody packages AutoRemesher into a pipeline yet. The two existing wrappers (a subprocess bridge and the wheel-based fork, both released *this week*) are retopo-only. Being first to wrap the newly-MIT engine in a merge→retopo→UV→bake pipeline is a genuine first-mover position, and the timing of this plan could not be better.

FastBake's differentiators, chosen to answer the four loudest complaint patterns found across competitor reviews: **reliability across Blender versions** (crash-isolated engine process, dual 4.x/5.x API paths), **no silent failures** (validate before every stage, always save to disk, plain-language errors), **a genuinely minimal UI** (three visible decisions, everything else in popups and presets), and **real per-stage progress with cancel**.

SECTION 2

Request analysis

All 25 requirements from the brief map cleanly onto seven pipeline stages plus four cross-cutting systems (naming, presets, progress, gating). Three points needed a design ruling rather than a literal translation:

Rulings on ambiguous or conflicting requirements

BRIEF SAYS	RULING
Roughness goes into the color texture's alpha *if the original material has no alpha*; checkbox "Roughness to alpha" (default on).	Cycles cannot bake alpha in one pass anyway, so alpha is always a post-bake numpy merge. Rule: if any source material drives Principled `Alpha` (linked socket or value < 1.0), the original alpha wins the alpha channel, roughness is written to a separate `T_…_roughness` texture *regardless of the checkbox*, and a one-line notice appears in the completion report. No hidden data loss.
Texture suffixes `_color`, `_normal`, `_roughness`; prefixes `T_`, `M_`.	Prefixes match Epic's official UE conventions exactly. The spelled-out suffixes match Substance's style; UE community linters expect letters (`_D`, `_N`, `_R`). Default stays as briefed (`_color`), but naming lives in an editable template with a bundled "Unreal (letter suffixes)" preset. Costs nothing, wins both audiences.
"Seams… try to not divide frontally."	"Front" is undefined for an arbitrary mesh. Default behavior uses view-independent hiding (ambient occlusion + concavity – seams settle in cavities and behind-facing areas on their own, the same trick ZBrush's UV Master uses). An optional "Protect view" toggle additionally penalizes edges visible from the current viewport or active camera. See stage 3.

One expectation to reset

Cycles exposes **no fine-grained progress during a single bake pass** — confirmed by Blender developers; the API simply doesn't surface it. FastBake therefore reports progress at stage-and-map granularity (e.g. "Baking 2/3 — normal"), which still gives 10–14 visible ticks across a full run. Everything else in the brief is implementable as

requested.

SECTION 3

Competitive landscape

TOOL	RETOPO	AUTO UV	PBR BAKE	PACKING	LIGHTMAP	PRI
SimpleBake (leader)	–	Smart UV only	✓	✓	✓	~$
BakeLab 2	–	–	✓	–	–	fre
Bake Wrangler	–	–	✓	✓	via passes	~$
Marmoset Toolbag 5	–	–	✓ GPU	✓	–	$3
Substance Painter 12.1	–	fallback	✓	✓ templates	–	sub
Exoside QuadRemesher	✓ best-in-class	–	–	–	–	$1
Soloware (2025)	✓	✓	N/AO/curv only	–	–	$9

Assetify 2.x	decimation	✓	✓	–	–	$3
Blender native bake	QuadriFlow	manual	one map at a time	–	–	fre
FastBake (planned)	**✓ quad**	**✓ seam-aware**	✓	✓	**✓ denoised**	tbc

* Blender-only Pro license; $59.90 Indie; the widely-quoted $139.90 is the all-softwares bundle (fact-check correction). Best current two-tool stack (QuadRemesher + SimpleBake) ≈ $80–130 and still has no UV bridge between them.

Pricing context if FastBake is sold: bake-only add-ons anchor at ~$20, the pipeline attempts at $35–99 (the $99 one was rejected by the community for thin evidence). **$30–50 perpetual undercuts the two-tool stack while staying above the toy tier**; free-on-extensions.blender.org is the alternative positioning (see §12).

SECTION 4

Architecture

Process model – three rules

1. **Blender's main thread is never blocked.** The pipeline driver is a modal operator plus a window timer; every tick it polls the running stage, updates progress properties, and tags the panel for redraw. Python `threading` is off-limits (documented crash source; extension moderators reject it) — heavy work goes to *processes*.
2. **The remesher runs out-of-process.** AutoRemesher has a fresh crash history on

malformed input (open issues #30/#35/#36). A native crash in-process would take Blender down with it; in a worker process it costs a stage retry. The worker streams the engine's native progress callback back through a pipe.

3. **Bakes run as Cycles background jobs** via `bpy.ops.object.bake('INVOKE_DEFAULT')`, sequenced by the state machine using `bpy.app.is_job_running('OBJECT_BAKE')` and the `object_bake_complete / object_bake_cancel` handlers. (Note: the 3.3 release notes misspell the last one as `object_bake_canceled` — the real symbol has no "ed". Fact-checked against Blender source.)

The state machine

```
IDLE → PREFLIGHT → APPLY TRANSFORMS → [MERGE] → RETOPO → UV → BAKE(normal → co
                                                              LIGHTMAP (own butl
```

- Stage functions live in a module-level queue (survives redraws; inspectable by the panel). Each stage: *enter* (optional confirmation gate) → *execute* (cancellable) → *exit* (`bpy.ops.ed.undo_push("FastBake: <stage>")`, provenance update).
- **Undo discipline:** one push at pipeline start plus one per stage — granular rollback and a clean stack (Blender's API docs warn that modifying data without undo steps corrupts the undo stack). Object references are re-resolved by name each stage; never cached across gates where the user might undo.
- **Failure = rollback + report.** Any stage error restores the pre-run undo point, unhides originals, and shows a popup with a copyable log. Originals are never destroyed — they are hidden at the end, tagged with a provenance property.
- **Review-each-stage checkbox** parks the machine in a WAITING state between stages; the panel shows the stage result summary with **Continue** / **Adjust…** / **Cancel**. "Adjust…" opens that stage's popup with its hidden parameters — this is where the "settings live in popups" philosophy pays off (§7).

SECTION 5

Stage-by-stage design

STAGE 0 Preflight

- **Nothing selected** → popup: "Select a mesh to bake." Non-mesh objects in the selection are ignored with a notice; zero meshes → same popup.
- **Already processed?** Each FastBake result carries an ID-property manifest (`obj["fastbake"]` : source names, settings hash, map list, run number, date). If the selection matches an existing result whose source hash is unchanged, popup: "*Cube_01 with 2 textures already exists — nothing to do.*" with **Bake Again** / **Cancel**. This catches the accidental re-press the brief calls out; source-mesh edits change the hash and re-enable processing.
- **Material analysis:** detect Principled Alpha usage (decides the roughness-packing rule, §2), detect metallic ≥ threshold (switches the color bake to the emission-rewire path, stage 4), count material slots.
- **Environment checks:** output directory writable (default `//textures/` , falls back to a popup asking for a path on unsaved files), disk space, Cycles available.
- **Apply transforms — last preflight step, before anything is duplicated:** rotation and scale are applied to the selected originals (`transform_apply` ; location kept, so nothing moves in the scene). Every downstream number assumes unit scale — BVH-measured cage distances, voxel sizing, MikkTSpace tangents — so applying up front removes a whole artifact class at the source. Multi-user mesh data is made single-user first (noted in the completion report); the pipeline-start undo step reverts it if the run is cancelled.

STAGE 1 Merge to watertight

runs only for multi-selection or broken single meshes

The watertight requirement exists *only* for the retopo stage — bakes ray against the original high-polys, so the merged mesh is a **scaffold**: silhouette and mid-scale features must survive; surface detail may resample. This is the same architecture as ZBrush's DynaMesh→ZRemesher and Simplygon's volumetric remesher — the production-proven answer.

Fast path	All shells closed, manifold, coherent normals → **Boolean self-union** (join duplicates, Boolean modifier, Operand = Collection, *empty* collection, Exact solver + Self-Intersection — a documented pure-bpy trick that dissolves interior faces while preserving geometry exactly). On Blender ≥ 4.5 with strictly-manifold shells, try the new **Manifold solver** first (the elalish/manifold library inside Blender: faster, guaranteed-manifold output).
Robust path	Default for sculpts/scans/kitbash and the automatic fallback: open or thin shells get a Solidify pass (thickness ≈ 2.5 × voxel), then **OpenVDB Voxel Remesh**. Watertight, manifold, intersection-free by construction — and it unions overlapping shells for free.
Auto-tuning	`voxel = diag/300` (Detail slider: Low diag/150 · Med diag/300 · High diag/600), clamped by memory guard `diag/1500` and coupled to the quad target (aim ~40× target quads worth of faces). Thin-shell estimate lowers voxel size so fine parts survive.
Validation	bmesh scan: zero boundary edges, zero non-manifold edges, per-component survival vs. input. Retry ladder: voxel × 0.7, then × 1.5, then surface the failure at the stage gate. **Loose-debris cleanup is manual**: the voxel remesher has *no* "Remove Disconnected" option (that belongs to the legacy modes — a fact-check correction), so tiny islands are removed by separate-loose-parts + volume threshold.
v2 upgrade	**CGAL Alpha Wrapping** (SIGGRAPH 2022) as primary combiner — provably watertight output from arbitrary soup, and it handles open cloth/planes as two-sided wraps with no Solidify heuristics. Needs a small self-built wheel (the official `cgal` PyPI wheels stop at Python 3.12 / Oct 2024, so they can't cover Blender 5.1+). Plus `manifold3d` (Apache-2.0, excellent wheels — but none for windows-arm64) as the exact-geometry fast path.

STAGE 2 Retopology – AutoRemesher 1.0

headline stage

Primary engine	`autoremesher_core` compiled as a **nanobind Python wheel** from the now-MIT, Qt-free core (the jakeblakeley fork is a working blueprint: 1–3 MB per platform, no Qt). Direct vertex-array I/O — no OBJ round-trip — and the C++ **progress callback** feeds the progress bar percent-by-percent through the worker pipe. Build with `-march=x86-64-v2` : the official 1.0 Linux build crashes with SIGILL on pre-AVX2 CPUs (open issue #33).
Dev-phase path	Before the wheel CI exists: bundle-or-locate the official CLI binaries, drive via list-form `subprocess` (`QT_QPA_PLATFORM=offscreen` on Linux). Identical pipeline semantics, coarser progress. This also remains the "bring your own binary" option.
Quad count	UI value maps to `--target-quads` (engine default 50 000; FastBake default 5 000). Output is approximate — read the actual count from the result; if it misses a hard budget by >25%, one retry with a scaled target.
Derived params	Adaptivity 1.0, sharp-edge 90°, smooth-normal 0° as defaults; an Organic / Hard-surface switch (the core's `ModelType`) lives in the Adjust popup only. No symmetry support in the engine — noted as a known gap, QuadRemesher pass-through covers it for owners.
Fallback chain	Engine crash/timeout → automatic **QuadriFlow** (`bpy.ops.object.quadriflow_remesh` , zero deps, weaker edge flow) with a notice at the stage gate. Optional pass-through to a user-installed **Exoside QuadRemesher** (its EULA forbids bundling/SDK use — detect-and-call only).

The merge stage doubles as crash prophylaxis: AutoRemesher's issue tracker is dominated by malformed-input crashes, and it always receives FastBake's validated watertight scaffold, never raw user geometry.

STAGE 3 UV unwrap – seams first, then SLIM

State-of-the-art review, condensed: Blender's Unwrap only cuts along seams you give it — **seam placement is the whole game**, and Blender doesn't automate it. The 2025–26 neural seam generators (SeamCrafter, ArtUV, MeshTailor) define paper SOTA but none has released usable code; the strongest academic few-island method (PartUV) is blocked by a non-commercial NVIDIA checkpoint. The production-usable pieces are: Blender's own **SLIM "Minimum Stretch" unwrap (4.3+)**, its rewritten concave-shape packer, a classical seam algorithm (Seamster, 2002) that remains the industry template, and ZBrush UV Master's philosophy — which Blender can replicate natively.

FastBake's advantage: it unwraps its own output. A fresh AutoRemesher mesh is clean, watertight, curvature-aligned quads — the easiest possible unwrapping input, with edge loops as natural seam rails.

Seam solver (v1)	Seamster-style, a few hundred lines of NumPy, sub-second at ≤20k faces: terminals at Gaussian-curvature extrema (+ genus handle loops so every surface becomes disk-cuttable), connected by Dijkstra over a cost graph: `cost(e) = len · e^(k_p·protect) · e^(−k_a·attract) · e^(−k_ao·(1−AO)) · g(dihedral) · loop_bonus` AO from BVH hemisphere sampling (no render needed); `g` makes concave creases cheap; `loop_bonus` keeps seams running along quad edge loops. AO + concavity is exactly how UV Master's "Attract from Ambient Occlusion" hides seams — view-independent "not frontal" by default.
Protect view	Optional toggle: rasterize visibility from the current viewport (or active camera) and multiply edge cost for visible edges — the literal "don't cut frontally" control.
Unwrap	`uv.unwrap(method='MINIMUM_STRETCH', iterations≈50, no_flip=True)` — SLIM balances angle *and* area distortion with guaranteed injectivity. Then measure per-face stretch; if a region exceeds threshold, add one relief cut through it and re-unwrap (max 2 rounds — the OptCuts idea at heuristic speed). This is the islands-vs-stretch balance the brief asks for: start with few cuts, add only where stretch demands.
Pack & QA	`average_islands_scale` → `pack_islands(shape_method='CONCAVE',`

`rotate_method='CARDINAL', margin_method='FRACTION', margin=4px/bake_res)` → overlap + flipped-face check as a hard gate. Fallback if the solver fails validation: `smart_project(angle_limit≈66°)`.

Guided seams (v1.x)

The ZBrush UV Master equivalent, all native: two weight-paint groups, **Attract** and **Protect**, feed `k_a / k_p` in the cost function — paint roughly where seams should and must-not go. UV Master's third mode (Density) already exists in Blender: SLIM's importance weights (`use_weights, weight_group`) give painted regions more UV area. Entered via "Paint seam guides…" in the UV stage's Adjust popup; nothing better exists to license as of July 2026, and this implementation is ~zero new solver work.

v2 option

Ministry of Flat as an external engine for users who install it (free license incl. commercial, CLI, *Windows-only*): export → `UnWrapConsole3.exe` → reimport → `seams_from_islands` → SLIM re-unwrap per island.

STAGE 4 Bake – normal, color, roughness

Cycles selected-to-active from the **original high-poly objects** (they hold the materials; the merged scaffold is retopo-only). Blender ≥ 4.4 is effectively required — 4.4 parallelized the CPU prep step that used to dominate multi-map bakes (100 s+ single-threaded at 4×4K).

Auto ray setup

A genuine differentiator: FastBake generated both meshes, so it can *measure* instead of guess. BVH query from every low-poly vertex to the high-poly surface → max distance + 50% = `cage_extrusion` , ×2 = `max_ray_distance` . The manual-tuning step that SimpleBake's own docs call the most common support issue simply disappears. Explicit cage object stays as an advanced escape hatch.

Normal

`type='NORMAL'` , tangent space (MikkTSpace), OpenGL Y+ default with a DirectX preset. **Triangulate-before-bake rule:** a fixed-diagonal Triangulate modifier is present during bake and kept for export — MikkTSpace tangents are only consistent for one triangulation, the classic invisible cause of shading seams. Saved

	16-bit PNG, Non-Color.
Color	`type='DIFFUSE', pass_filter={'COLOR'}` — pure albedo, no lighting, noise-free at 2 samples. Metallic materials bake black in the diffuse pass, so preflight-flagged materials switch to the **emission-rewire trick** (Base Color → temporary Emission shader → `type='EMIT'`) — the canonical workaround, also used for baking source alpha when needed.
Roughness	`type='ROUGHNESS'`, 1–2 samples, Non-Color — baked always; whether it lands in alpha or its own file is decided by the packing rule (§2).
Quality	Bake at 2× resolution and downscale (SSAA) by default — Cycles has no anti-aliasing on UV island edges regardless of samples (open bug, brecht: "known issue"). Margin 16 px/1K equivalent, `ADJACENT_FACES`. Data passes: 2 samples, adaptive sampling off, GPU on.
Blender 5.0 rule	Since 5.0 Cycles bakes only to image nodes that are *selected and active* (breaking change). FastBake unconditionally deselects all nodes and selects+activates its target node in **every** material slot — backward-compatible with 4.x.

STAGE 5 Pack & save

- Roughness→alpha is a numpy merge: `pixels.foreach_get / foreach_set` (orders of magnitude faster than list access; instant at 4K), `col[3::4] = rough[0::4]`, then `alpha_mode='CHANNEL_PACKED'` so Blender neither premultiplies nor displays roughness as transparency. Roughness stays linear in alpha — no gamma.
- The SSAA downscale happens in the same numpy pass (better filtering than `image.scale()`).
- **Everything is saved to disk immediately** (default `//textures/`) — the "unsaved bake silently lost on close" failure class from native Blender is designed out. PNG default; 16-bit for normals.

STAGE 6 Material build

`M_<Base>` (numbered with the run counter): Principled BSDF; color texture → Base Color (sRGB); its alpha → Roughness via the image node's Alpha socket (or the separate roughness texture if packing was off / overridden); normal texture (Non-Color) → Normal Map node → Normal. Assigned to the new mesh; result object gets the provenance manifest; originals are hidden into a `FastBake_Sources` collection, never deleted.

STAGE 7 Lightmap (separate button)

Settings popup	Per the brief, gamma is adjustable *before* bake: the Lightmap button opens a small dialog — size (default 1024), samples (default 64), **gamma** (default 1.0), denoise (default on). Values persist in the preset.
Bake	`type='DIFFUSE', pass_filter={'DIRECT','INDIRECT'}` — light without albedo, so an engine can multiply lightmap × color. Float buffer, saved EXR/HDR. Runs on FastBake UVs by default (guaranteed non-overlapping); a dedicated UV2 via the MIT `xatlas` wheel is the advanced option (xatlas is the industry standard for exactly this; its many-small-charts weakness doesn't matter for lightmaps).
Denoise	Cycles can't denoise non-Combined bake passes (confirmed by the Cycles lead dev), so denoising is post-bake: a scripted compositor graph — Image → **Denoise (OIDN)** → optional Gamma → output — with zero external dependencies. The Lightmapper add-on proves the exact pattern. Dual API path required: Blender 5.0 replaced `scene.node_tree` with `scene.compositing_node_group` and moved Gamma to `ShaderNodeGamma`.
Naming	`T_<Base>_lightmap`, same run-counter numbering. Note in docs: engines expect linear lightmaps — gamma ≠ 1.0 is a creative/preview choice.

SECTION 6

Naming engine

One module owns every name. *Base* = active object's name stripped of trailing `_NN` / Blender's `.NNN`; for multi-selections the active object names the asset. One **run counter** is computed per bake (scanning existing blend data *and* the output folder) and shared by all textures and the material of that run, so a re-bake never produces a mixed set like `T_Rock_color_01` + `T_Rock_normal_02`. The mesh runs one number ahead by design — the brief starts meshes at `_01` on the first bake, while textures and material gain numbering only from the second bake on:

ARTIFACT	FIRST BAKE	SECOND BAKE
Mesh	`Rock_01`	`Rock_02`
Color (roughness in alpha)	`T_Rock_color`	`T_Rock_color_01`
Normal	`T_Rock_normal`	`T_Rock_normal_01`
Roughness (only if packing off / alpha conflict)	`T_Rock_roughness`	`T_Rock_roughness_01`
Lightmap	`T_Rock_lightmap`	`T_Rock_lightmap_01`
Material	`M_Rock`	`M_Rock_01`

- **Edge cases:** source named `Cube_01` → base `Cube`, result `Cube_02` (strip then renumber — never `Cube_01_01`). Only local IDs count toward collisions; linked-library data is ignored.
- **Template-driven:** the scheme above is the default template (`T_{base}_{map}{run}`). A bundled "Unreal" preset switches to letter suffixes (`_D/_N/_R`) — Epic's official page mandates the `T_` / `M_` prefixes; the letter suffixes are dominant

community practice (Allar / Looman style guides).

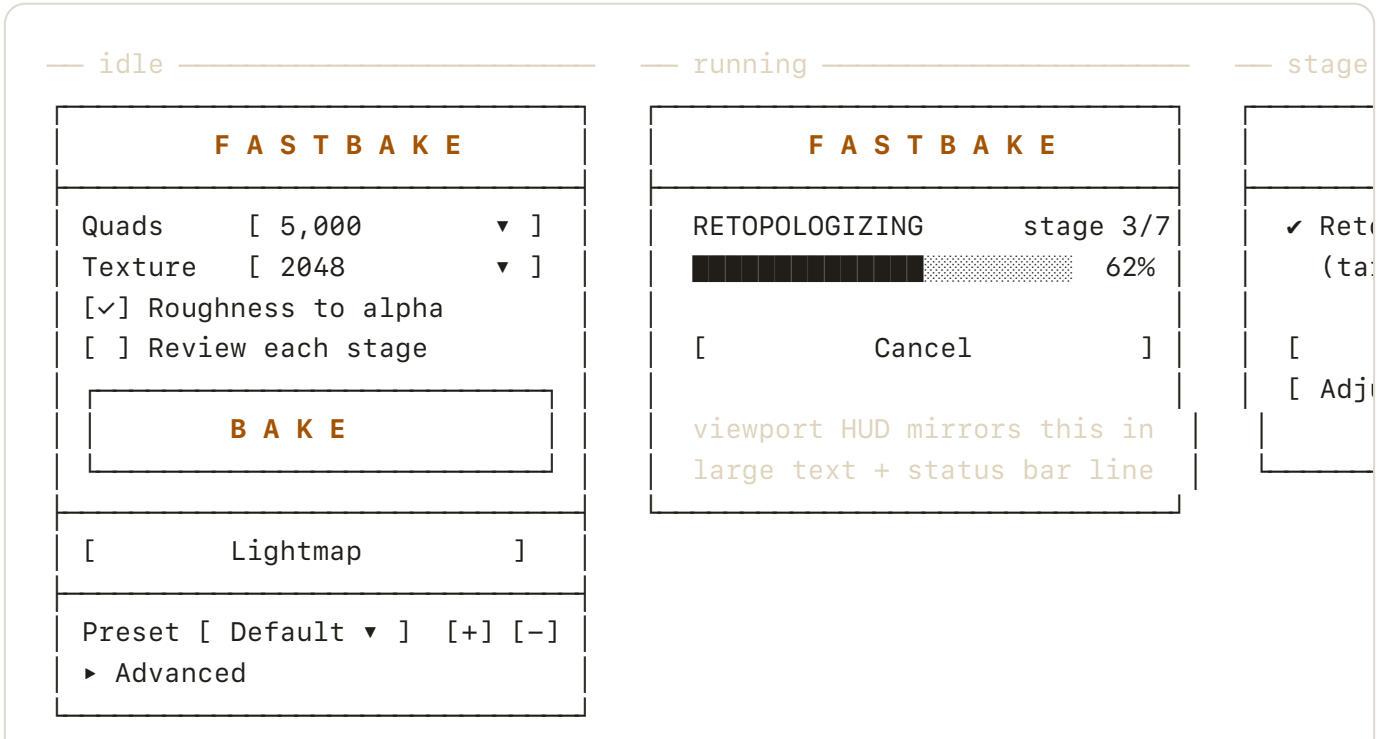
- **Provenance:** result object gets `obj["fastbake"] = {version, sources, settings_hash, run, quads, tex, maps, date}`; each source gets `fastbake_source_of`. This drives re-bake detection (§5 stage 0) and travels into engine exports as glTF extras.

SECTION 7

UI & UX

FastBake lives as its own tab in the **3D Viewport sidebar (N-panel)**, category "FastBake". Philosophy from the brief, sharpened by the competitor teardown: SimpleBake's one panel grew to dozens of toggles and its own users call it intimidating. FastBake's default view shows **one settings group capped by the hero BAKE button, a Lightmap section, and the preset row** — everything else is a popup at the moment it's relevant, or lives inside a preset.

The panel – three states



- **Header:** Blender panels can't change font size (`UILayout` has no per-widget type scale — verified), so "FastBake" reads large via a `scale_y ≈ 1.6` spaced-letter label row plus the panel category. The truly big type lives in the viewport HUD.
- **BAKE button:** full-width row, `scale_y ≈ 2.4` — the biggest thing in the panel by a factor of two, and it **closes the settings group** (same layout column, no separator above), so the four settings and their trigger read as one unit: what you set is what BAKE runs. `depress=True` while running. Lightmap sits in its own section below the group.
- **Progress, large and visible:** three synchronized channels. (1) The panel swaps its settings block for a stage label + full-width `layout.progress()` bar (native widget, Blender 4.0+), redrawn from the pipeline timer. (2) The workspace status bar text. (3) A **viewport HUD** — a `draw_handler` with `blf` drawing stage name + percent in genuinely large type in the 3D view (the SimpleBake pattern, and the only way to get big text). HUD on by default, toggleable.
- **Popups:** `invoke_confirm` for gates and warnings ("Nothing selected", "Already baked", "Source uses alpha — roughness saved separately"); `invoke_props_dialog` for the Lightmap settings and each stage's Adjust dialog. Per-stage gating uses the panel-state-machine pattern (buttons in the panel, not stacked dialogs — dialogs can't chain reliably from `execute()`).
- **Advanced (collapsed):** output directory, margin, SSAA, normal-map Y convention, engine choice (AutoRemesher/QuadriFlow/QuadRemesher-if-installed), merge detail, seam-guide painting entry, lightmap UV2 toggle. Nothing here is required for the default run.

SECTION 8

Presets

Blender's native preset system stores executable `.py` files and has **no import/export**

— wrong tool for the brief and a moderation flag (exec-based). FastBake uses a JSON layer: the settings PropertyGroup serializes via `bl_rna` introspection; presets live in `bpy.utils.extension_path_user(…/presets)` (upgrade-safe, removed on uninstall); Export/Import are two file-browser operators moving plain `.json`. Scene-level settings (a `PointerProperty`) save with the .blend; machine-level defaults (output root, HUD toggle, review default) live in AddonPreferences.

```
{
  "fastbake_preset": 1,
  "name": "Props 2k",
  "quads": 5000,
  "tex_size": 2048,
  "roughness_to_alpha": true,
  "review_stages": false,
  "naming":   { "template": "T_{base}_{map}{run}", "material": "M_{base}{run}'
  "advanced": { "margin_px": 16, "ssaa": 2, "normal_y": "OPENGL",
                "engine": "AUTOREMESHER", "merge_detail": "MED",
                "lightmap": { "size": 1024, "samples": 64, "gamma": 1.0, "denc
                "output_dir": "//textures/" }
}
```

Ships with three factory presets: **Props 2k** (5k quads / 2048), **Hero 4k** (20k / 4096), **Mobile 1k** (1.5k / 1024, letter-suffix naming).

SECTION 9

Packaging & platform

Format	Blender extension (`blender_manifest.toml`, schema 1.0.0), `blender_version_min = "4.5.0"`. Rationale: 4.2 LTS dies this month (July 2026); 4.5 LTS is supported to July 2027; and the bake-prep parallelization (4.4) plus the Manifold boolean solver (4.5) land inside the floor. Supported: 4.5 LTS, 5.0, 5.1, 5.2 LTS.
Dual API paths	Isolated in one `compat.py`: compositor tree creation (`scene.node_tree` → `compositing_node_group`), `ImageFormatSettings.media_type` -

	before- `file_format`, UV selection API, boolean solver rename FAST→FLOAT, no dict-style access to registered props (all Blender 5.0 breaking changes, verified against release notes).
License	Add-on code GPL-3.0-or-later (required by `bpy` and extensions.blender.org). AutoRemesher core MIT, Geogram BSD-3, Eigen MPL-2.0, TBB Apache-2.0 — all GPL-compatible; each credited in the manifest `copyright` list. GPL permits selling; only source availability to buyers is required.
Binary wheels	`autoremesher_core` published to PyPI (the platform requires wheels "bundled unmodified from PyPI"), bundled per platform via `--split-platforms`. Matrix: {windows-x64, macos-arm64, macos-x64, linux-x64} × {cp311 for 4.5–5.0, cp313 for 5.1+} = 8 builds per release via cibuildwheel CI, `-march=x86-64-v2`. Optional wheels later: `xatlas` (lightmap UV2), `manifold3d` (v2 merge). windows-arm64 deferred (no manifold3d wheels there; niche).
Review compliance	No network permission at all; `files` permission with justification; list-form subprocess args only; temp/cache exclusively in `extension_path_user`; no threading (processes only); no runtime downloads (kills any "fetch engine on first run" idea — binaries ship in the zip or not at all).

SECTION 10

Risks & mitigations

RISK	SEVERITY	MITIGATION
AutoRemesher 1.0 is **three days old**: open crash issues (#30/#35/#36), hardening PR in flight, no independent quality benchmarks yet	High	Pin to a vetted commit; out-of-process execution (crash ⇒ retry/fallback, not a Blender crash); always feed validated watertight input; QuadriFlow auto-fallback; expect to rebase onto 1.0.1 quickly – and upstream fixes (it's MIT and active again)
Wheel build matrix (8 binaries/release) is real	High	cibuildwheel GitHub Actions from day one; the jakeblakeley fork is a working scikit-build-core

maintenance cost		blueprint; CLI-subprocess path stays as escape hatch
Thin sheets / open shells break the voxel merge path	Med	Open-shell detection → auto-Solidify ladder; per-component survival validation; alpha-wrap upgrade in v2 makes the problem vanish
Blender 5.x API churn (the #1 competitor complaint is version breakage)	Med	Single `compat.py` seam; CI smoke-test against 4.5/5.0/5.1/5.2-beta nightly matrix; no private API use
Seam quality on hard-surface meshes (v1 solver is curvature/AO-driven)	Med	Sharp-edge dihedral term already favors crease seams; guided paint mode (v1.x) as manual override; Ministry of Flat option (v2) for Windows users
No within-pass bake progress (API limitation)	Low	Per-map granularity + stage weights still yields 10–14 visible ticks; set expectation in docs
Exoside EULA / Ministry of Flat licensing if integrated carelessly	Low	Detect-and-call user installs only; never bundle; MoF free license covers commercial use but not SaaS — irrelevant for a local add-on

SECTION 11

Roadmap

MILESTONE	SCOPE	EXIT CRITERION
`M0 · skeleton`	Extension scaffold, settings model, naming engine + tests, JSON presets, panel UI with mock pipeline, preflight popups	UI walkthrough works end-to-end with stub stages
`M1 · merge`	Diagnosis, boolean fast path, voxel robust path, auto-tuning, validation + retry ladder	20-mesh torture set (open shells, self-intersections, kitbash) all produce validated watertight

		scaffolds
`M2 · retopo`	CLI-subprocess integration first; wheel CI (cibuildwheel, 8 targets) in parallel; QuadriFlow fallback; cancel + crash recovery	Remesh with live progress + cancel on all 3 OSes; kill −9 of worker leaves Blender healthy
`M3 · UV`	Seam solver (curvature+AO+concavity Dijkstra, genus cuts), SLIM unwrap, relief-cut loop, pack, QA gates	Benchmark set beats Smart UV on island count *and* stretch; zero overlaps/flips
`M4 · bake+pack`	Bake state machine, BVH auto-ray-distance, emission rewires, SSAA, numpy pack, material builder, provenance	Full BAKE run on 4.5 LTS and 5.1; re-bake produces correctly numbered `_01` family; re-press correctly refuses
`M5 · lightmap`	Settings dialog, DIFFUSE D+I bake, compositor OIDN denoise (dual 4.x/5.x path), gamma, EXR out	Denoised lightmap visually clean at 64 samples on demo scene
`M6 · polish`	Stage gates + Adjust popups, viewport HUD, error rollback, docs	Review-mode run with an undo at a gate doesn't corrupt state
`M7 · release`	QA matrix (4.5/5.0/5.1/5.2 × win/mac/linux), factory presets, demo assets, listing	v1.0 shipped to chosen channel
`v1.x`	Guided seam painting (attract/protect/density), UE/Unity naming presets, ORM packing preset, QuadRemesher pass-through	—
`v2`	Alpha-wrap merge wheel, manifold3d fast path, Ministry of Flat bridge, symmetry-aware seams; re-evaluate neural seam models (SeamCrafter/MeshTailor) when code+licenses ship	—

SECTION 12

Open decisions

None of these block M0–M2; defaults are chosen so work can start immediately.

1 · Distribution channel

extensions.blender.org (free, wheels-via-PyPI mandatory, maximum reach) vs. Superhive/Gumroad (paid ~$30–50, looser packaging rules, revenue).

Default: build to extensions.blender.org standards either way – it's the stricter superset.

2 · Suffix style default

Spelled-out `_color/_normal` (as briefed, Substance-like) vs. UE letter style `_D/_N`.

Default: as briefed; "Unreal" preset bundled. Pure template swap.

3 · "Front" definition for seam protection

AO/concavity already hides seams view-independently. Should the optional Protect-view toggle use the current viewport, the active camera, or −Y?

Default: off; when on, current viewport at press time.

4 · macOS Intel support

Official AutoRemesher 1.0 binaries are arm64-only, but our own wheel build can target macos-x64 at the cost of one more CI target.

Default: include macos-x64 (Blender still supports it).

Method. This plan was produced from six parallel research reports (AutoRemesher status, UV/seam state of the art, watertight merging, Blender bake APIs, competitor teardown, extension-platform rules), each independently fact-checked against primary sources on 2026-07-09 (48 load-bearing claims: 43 confirmed, corrections absorbed). Notable corrections that changed the plan: voxel remesh has no Remove-Disconnected option (loose-parts cleanup added); the bake-cancel handler is `object_bake_cancel`, not the release-notes' `…_canceled`; official CGAL wheels stop at Python 3.12 (self-built wheel planned); QuadRemesher's Blender-only price is $109.90, not $139.90; UE letter suffixes are community convention, not Epic's page.

Key sources: github.com/huxingyi/autoremesher · docs.blender.org (manual + API, 4.3–5.1 release notes) · developer.blender.org (extension moderation, release notes) · extensions.blender.org ToS · CGAL Alpha_wrap_3 manual · Sheffer & Hart, "Seamster" (2002) · Maxon UV Master docs · SimpleBake / BakeMaster / The Lightmapper docs & source · Epic asset-naming conventions.